

Campus Portal Mini-System

Spring Boot IoC & DI with React Frontend

Student Name: Saad Mughal

Student ID: F2023-009

Course: Web Engineering

Instructor: Sir Nouman Ali

Submission Date: November 10, 2025

1. Project Overview

This assignment demonstrates the implementation of a Campus Portal system using Spring Boot's Inversion of Control (IoC) and Dependency Injection (DI) principles, integrated with a React frontend. The project showcases loosely coupled architecture, layered design patterns, and full-stack integration.

Technologies Used

Backend Framework	Spring Boot 3.5.7
Java Version	Oracle JDK 17
Build Tool	Maven
Database	MySQL 8.x
Frontend Framework	React 18

UI Library	Bootstrap 5 (React-Bootstrap)
Development IDE	IntelliJ IDEA & WebStorm
API Testing	Postman

2. Dependency Injection Types Implementation

2.1 Field Injection

Field injection uses the `@Autowired` annotation directly on class fields. Spring injects dependencies after object instantiation.

Advantages: Concise syntax, less boilerplate code.

Disadvantages: Cannot create immutable fields, harder to test, hides dependencies.

2.2 Constructor Injection

Constructor injection passes dependencies as constructor parameters. This is the recommended approach in Spring.

Advantages: Enables immutability, easier unit testing, explicit dependency declaration, mandatory dependencies enforced at compile time.

Disadvantages: More verbose for multiple dependencies.

2.3 Setter Injection

Setter injection uses `@Autowired` on setter methods to inject dependencies after object creation.

Advantages: Allows optional dependencies, enables re-configuration after instantiation.

Disadvantages: Mutable dependencies, less explicit than constructor injection.

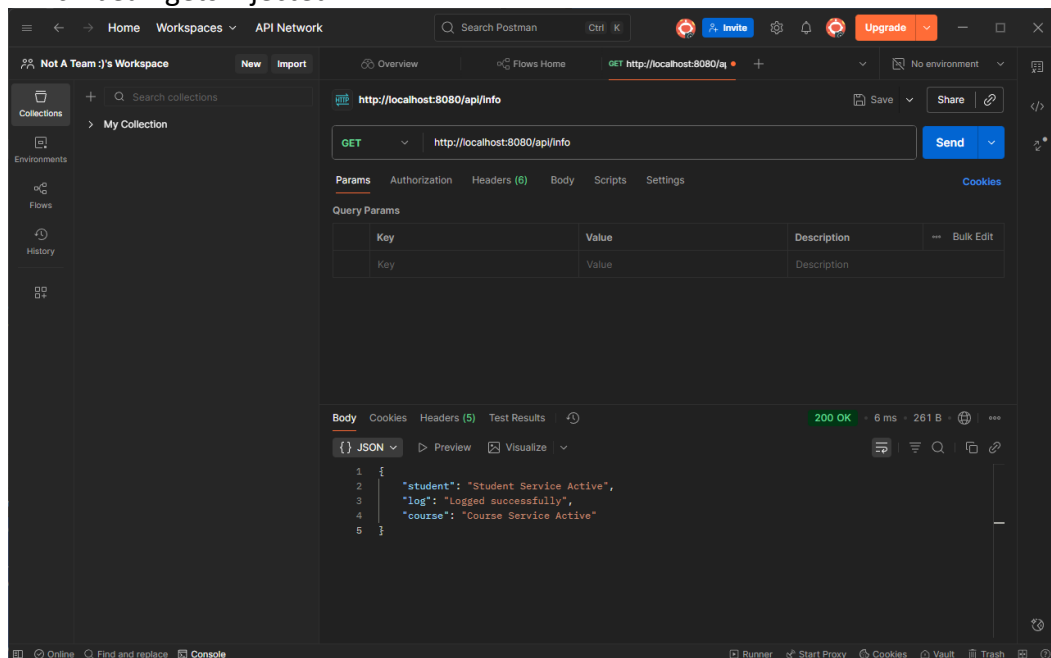
3. Task Implementation Details

Task 1: Basic Service Injection

Components:

- `MessageService` interface with `getMessage()` method
- `StudentMessageService` implementation returning "Welcome Student to Campus Portal"
- `AdminMessageService` implementation returning "Welcome Admin to Campus Portal"
- `WelcomeController` demonstrating all three injection types

Bean Selection: Used `@Primary` and `@Qualifier` annotations to control which bean gets injected.



Screenshot: Task 1 POSTMAN test.

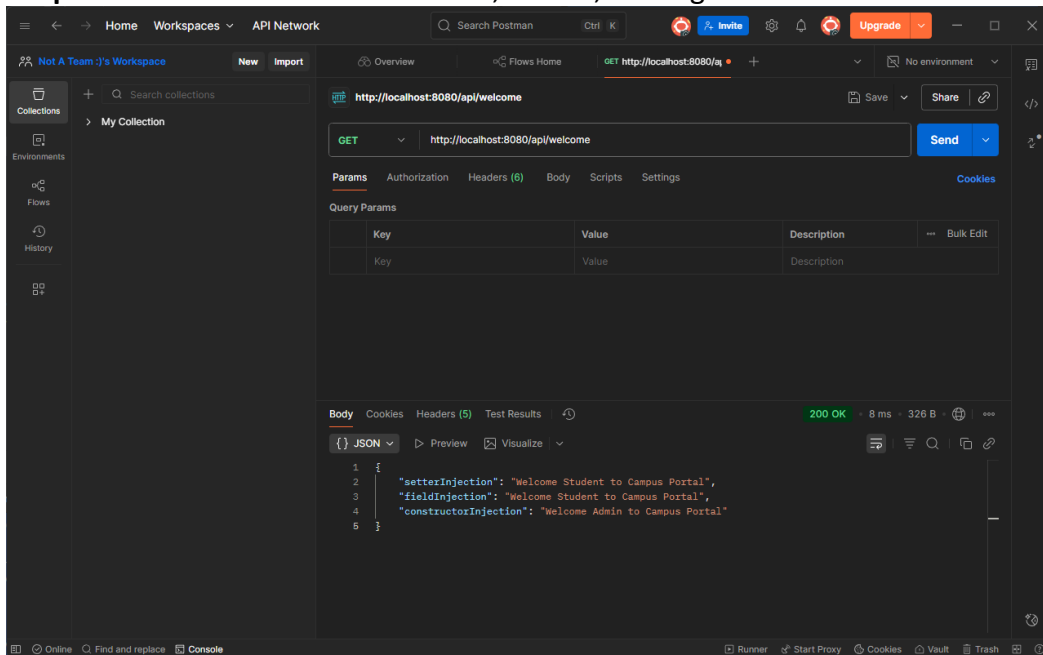
Task 2: Layered Architecture with DI

Architecture:

- `LoggerService`: Common logging service with `log(String message)` method
- `StudentService`: Handles student-related operations, depends on `LoggerService`
- `CourseService`: Handles course-related operations, depends on `LoggerService`

- `InfoController`: REST controller exposing `/api/info` endpoint

Response Format: JSON with student, course, and log status information.

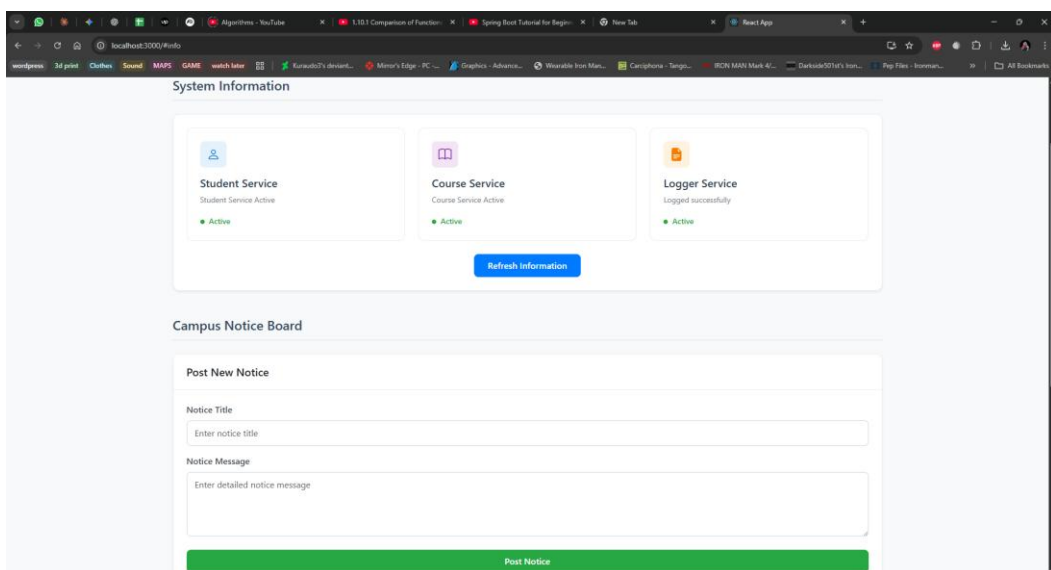


Screenshot: screenshot-task2-postman-api-info.png

Task 3: React Frontend Integration

Implementation:

- React component fetches data from `/api/info` endpoint on mount
- Displays JSON response in styled Bootstrap grid
- Includes "Refresh Info" button to re-fetch data
- Uses React hooks (`useState`, `useEffect`) for state management



Screenshot: screenshot-task3-react-info-display.png

Task 4: Profile-Based Bean Injection

Components:

- `DatabaseConfig` interface
- `MySQLDatabaseConfig`: Active in dev profile (`@Profile("dev")`)
- `PostgreDatabaseConfig`: Active in prod profile (`@Profile("prod")`)
- `/api/dbinfo` endpoint showing active database configuration

Configuration: Uses separate `application-dev.properties` and `application-prod.properties` files.

Task 5: Campus Notification System

Backend Components:

- `Notice` entity with id, title, message, and timestamp fields
- `NoticeRepository`: JPA repository for database operations
- `NotificationService`: Business logic for notice management
- `NotificationController`: REST endpoints for notice operations

API Endpoints:

- `POST /api/notice`: Create new notice (request body: title, message)
- `GET /api/notice`: Retrieve all notices

Frontend Components:

- `NoticeBoard` component displaying all notices in Bootstrap cards
- Form with title and message inputs for creating new notices
- Real-time state update and re-rendering on form submission

Home Workspaces API Network Search Postman Ctrl K Invite Upgrade

Not A Team's Workspace New Import Overview Flows Home POST http://localhost:8080/ap/notice No environment

Collections + Search collections My Collection

Environments

Flows

History

POST http://localhost:8080/ap/notice Save Share Send

Params Authorization Headers (8) Body Scripts Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "title": "Welcome to Campus Portal",
3   "message": "This is the first notice for all students. Classes will begin next Monday."
4 }
```

Body Cookies Headers (8) Test Results 201 Created 155 ms 494 B

JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Notice added successfully",
4   "notice": {
5     "id": 1,
6     "title": "Welcome to Campus Portal",
7     "message": "This is the first notice for all students. Classes will begin next Monday.",
8     "createdAt": "2025-11-09T19:37:55.7857781"
9   }
10 }
```

Online Find and replace Console Runner Start Proxy Cookies Vault Trash

Home Workspaces API Network Search Postman Ctrl K Invite Upgrade

Not A Team's Workspace New Import Overview Flows Home GET http://localhost:8080/ap/notice No environment

Collections + Search collections My Collection

Environments

Flows

History

GET http://localhost:8080/ap/notice Save Share Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

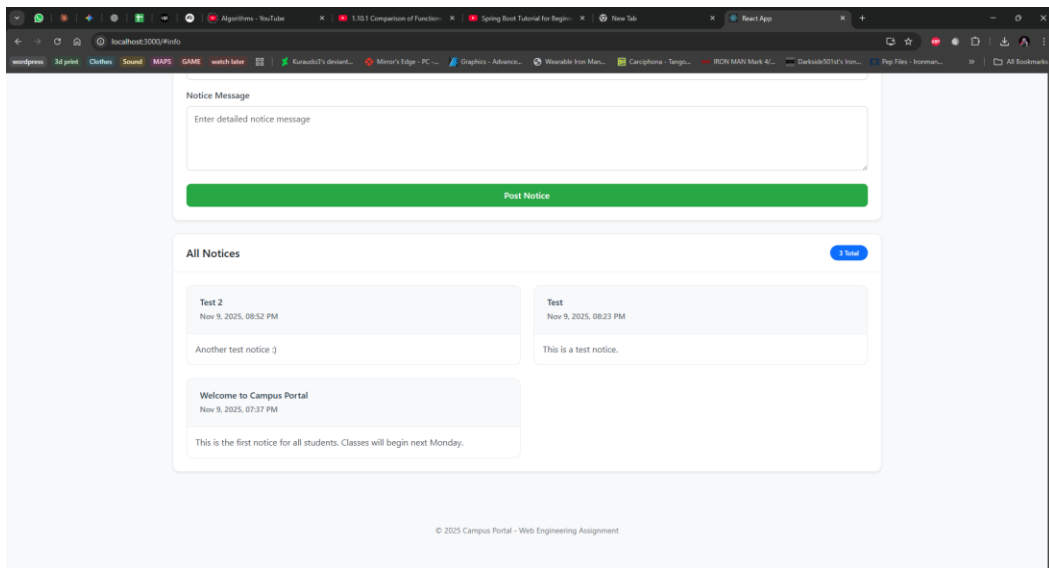
Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (8) Test Results 200 OK 159 ms 255 B

JSON Preview Visualize

```
1 []
```

Online Find and replace Console Runner Start Proxy Cookies Vault Trash



Screenshot: screenshot-task5-postman-post-notice.png

Screenshot: screenshot-task5-postman-get-notice.png

Screenshot: screenshot-task5-react-noticeboard.png

4. Class Relationships and Dependencies

Task 1 Relationships

- `WelcomeController` depends on `MessageService` interface
- `StudentMessageService` implements `MessageService`
- `AdminMessageService` implements `MessageService`
- Controller uses different injection methods for demonstration purposes

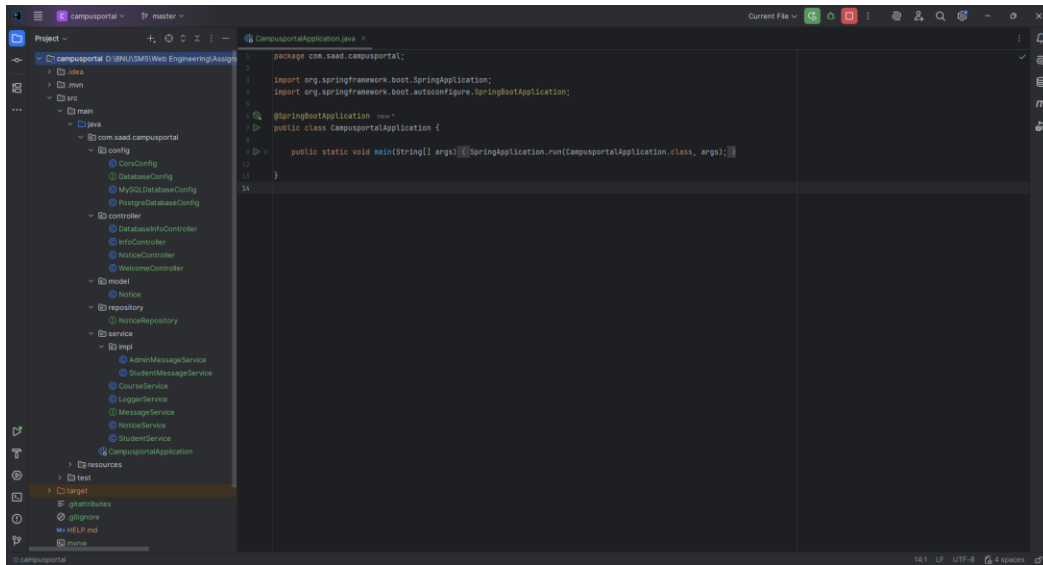
Task 2 Relationships

- `StudentService` depends on `LoggerService`
- `CourseService` depends on `LoggerService`
- `InfoController` depends on both `StudentService` and `CourseService`
- Demonstrates loose coupling through shared service dependency

Task 5 Relationships

- `NotificationController` depends on `NotificationService`
- `NotificationService` depends on `NoticeRepository`
- `NoticeRepository` extends `JpaRepository` for database operations

- Follows standard three-layer architecture: Controller → Service → Repository



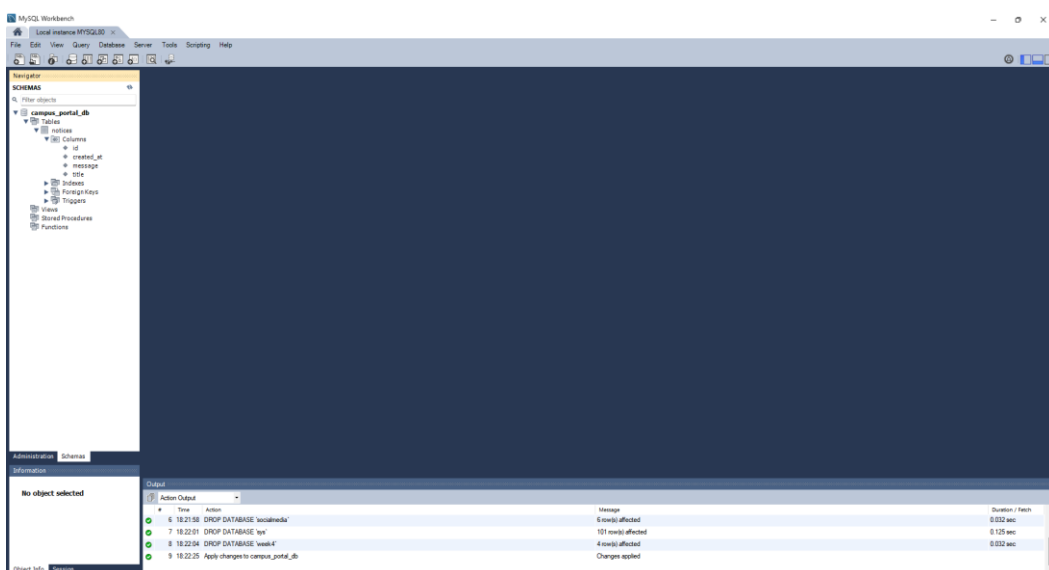
Screenshot: screenshot-project-structure.png

5. Database Configuration

Database: MySQL (campus_portal_db)

JPA Configuration: Hibernate auto-DDL enabled for automatic schema generation

Connection Details: Configured in application.properties with MySQL JDBC driver



Screenshot: screenshot-mysql-notices-table.png

6. Testing and Validation

All API endpoints were tested using Postman to ensure proper functionality:

- GET requests verified correct data retrieval
- POST requests validated data persistence to database
- Profile-based configuration tested by switching active profiles
- Frontend-backend integration tested through browser

7. Key Learning Outcomes

- Understanding of Spring's IoC container and how it manages bean lifecycle
- Practical implementation of different dependency injection types and their use cases
- Building loosely coupled, maintainable code through interface-based design
- Implementing layered architecture with proper separation of concerns
- Profile-based configuration for environment-specific bean activation
- Full-stack integration between Spring Boot REST APIs and React frontend
- Using Spring Data JPA for database operations with minimal boilerplate

8. Future Enhancements

- Implement Spring Security for authentication and authorization
- Add JWT-based token authentication for API endpoints
- Restrict POST /api/notice endpoint to admin users only
- Add input validation and error handling
- Implement pagination for notice listing
- Add edit and delete functionality for notices

9. Conclusion

This assignment successfully demonstrates the core concepts of Spring Boot's Inversion of Control and Dependency Injection principles. The implementation showcases how loosely coupled components can be wired together using Spring's IoC container, creating a maintainable and scalable application architecture. The integration with React frontend demonstrates practical full-stack development skills and the ability to build complete web applications.